Code Sentinel
System Design Document

1. Introduction

Code Sentinel is a multi-agent automated code review platform. Five independent specialist agents Security, Style, Performance, Logic and Documentation analyse a pull request or an on-demand diff in parallel. An orchestrator collects their findings, removes duplicates, ranks them by severity and then by confidence, attaches similar issues seen in the past, and returns one combined report.

The same report is delivered through two surfaces that must never disagree: a GitHub App (Check Run plus inline suggestions) and a Next.js web dashboard (history, reports and configuration).

This document covers the system architecture, the database schema, the class, component, state and sequence diagrams, the REST API contracts, and the UI/UX wireframes.

1.1 Design principles

- Partial is a success state one slow or failing agent must not block the whole review.
- Nothing reaches a branch on its own an AI-suggested fix is committed only on an explicit developer action.
- Deterministic style fixes (ESLint / Prettier / Ruff / Black) involve no LLM and are safe to apply directly.
- Every read is scoped to one organization (tenant isolation).
- Agents are stateless and independently deployable behind one shared REST contract, so any one can be scaled, tested or reused on its own.

2. System Architecture

The system is organised into five layers. Clients reach only the API Gateway; nothing downstream is exposed to the internet.

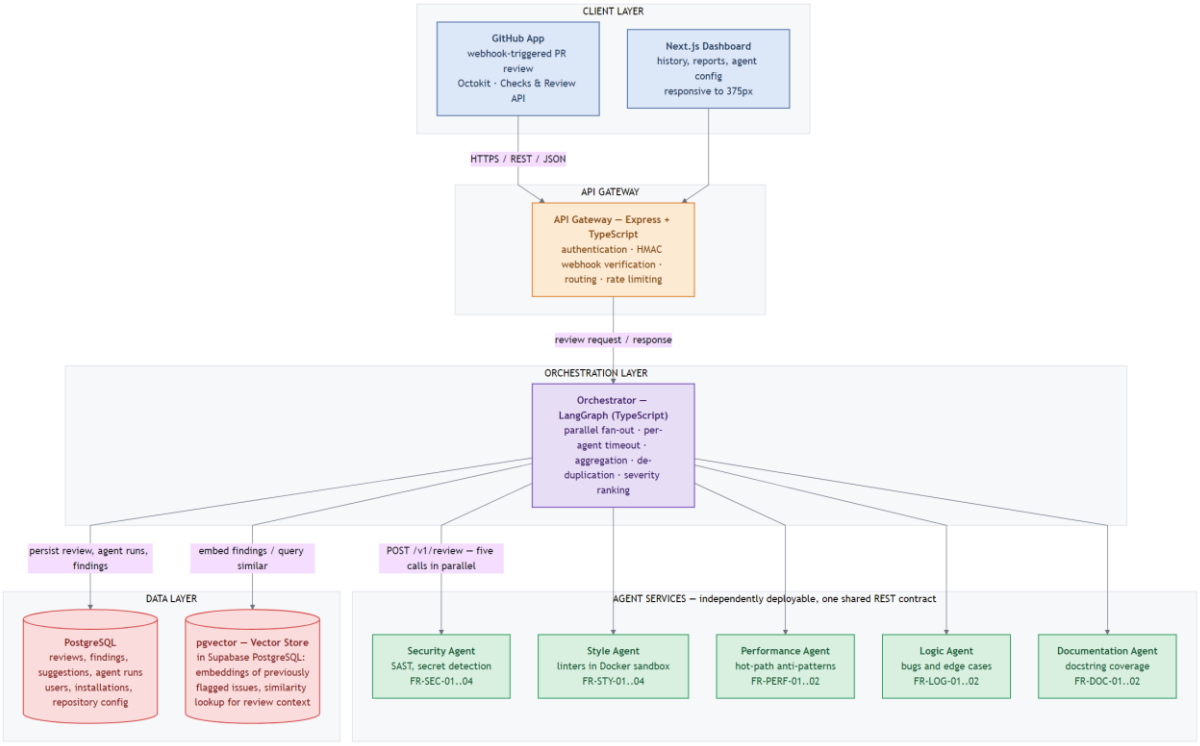


Figure 2.1 Layered architecture

- Client layer GitHub App (webhook-triggered, Octokit / Checks & Review API), Next.js dashboard (responsive to 375px).
- API Gateway Express + TypeScript. Authentication, HMAC-SHA256 webhook verification, routing, per-installation rate limiting.
- Orchestration layer LangGraph (TypeScript). Parallel fan-out, per-agent timeout, aggregation, de-duplication, severity ranking, vector-store context, Groq and Gemini quota budgets and large-diff planning.
- Agent services five independently deployable services, one shared REST contract (POST /v1/review).
- Data layer PostgreSQL (reviews, findings, suggestions, agent runs, users, installations, configuration) and pgvector in the same Supabase database (embeddings of previously flagged issues for similarity lookup).

2.1 Component view

Internal components of the gateway, orchestrator, a single agent service and the persistence layer.

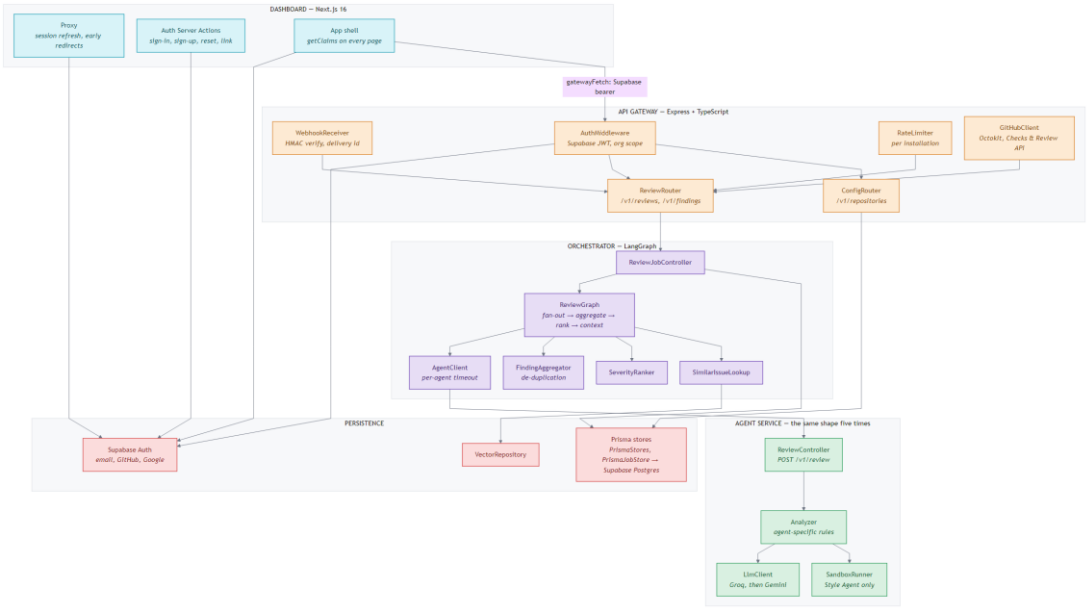


Figure 2.2 Component diagram

2.2 Deployment view

One web service per component on Render, the dashboard on Vercel, and PostgreSQL with pgvector and Auth on Supabase; Grafana Cloud collects metrics. The five agent services hold no state, so a slow agent is scaled out rather than tuned around. No agent talks to another agent, or to the database, directly.

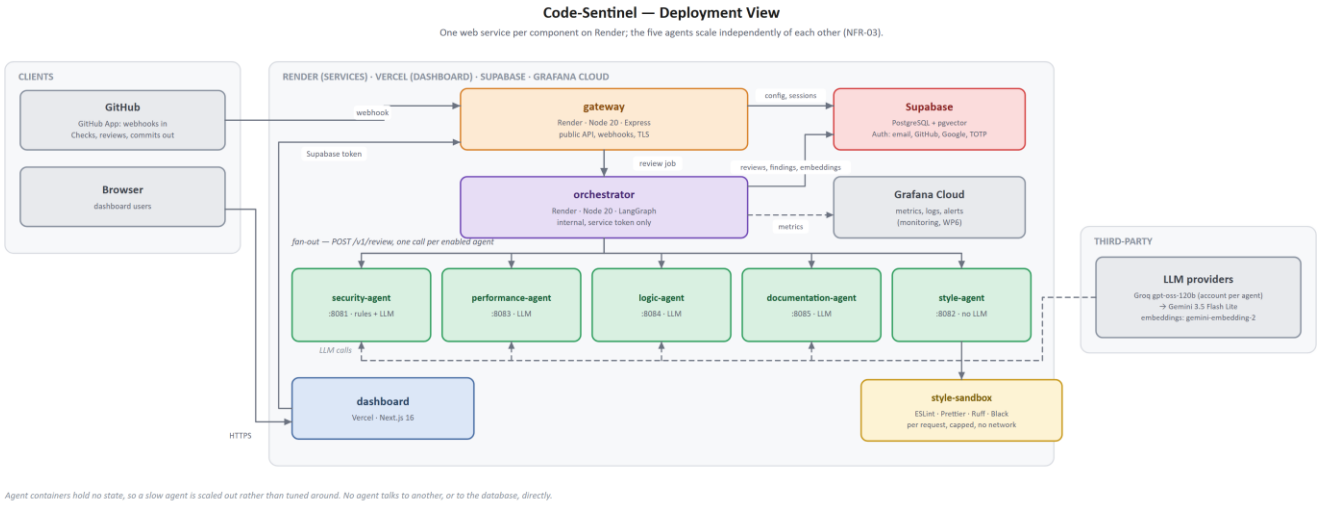


Figure 2.3 Deployment view

3. Database Schema (ER Diagram)

Target: PostgreSQL 15+ on Supabase, with the pgvector extension. The schema falls into two logical groups identity / tenancy and the review pipeline. Vectors live in pgvector next to the findings (finding_embeddings), and finding_similarities keeps the matches, so a finding can be traced both ways.

3.1 Enumerated types

Type	Values
org_role	admin, member
agent_kind	security, style, performance, logic, documentation
review_trigger	github_pull_request, dashboard_manual
review_status	queued, running, completed, partial, failed
check_conclusion	pass, fail, neutral
agent_run_status	pending, running, succeeded, timed_out, failed, skipped
severity	critical, warning, info
fix_kind	deterministic, ai_suggested
suggestion_state	proposed, accepted, rejected, superseded

3.2 Identity and tenancy

- organizations / users / organization_members a GitHub organization, its users, and their admin or member role.
- github_installations the GitHub App installation. The installation token is never stored; it is minted per request from the App private key.
- sessions not stored here: sign-in sessions live in Supabase Auth, and the gateway checks on each request that the session is still active.
- repositories / repository_settings / repository_agent_config one repository, its thresholds and timeouts, and one enable flag per agent.

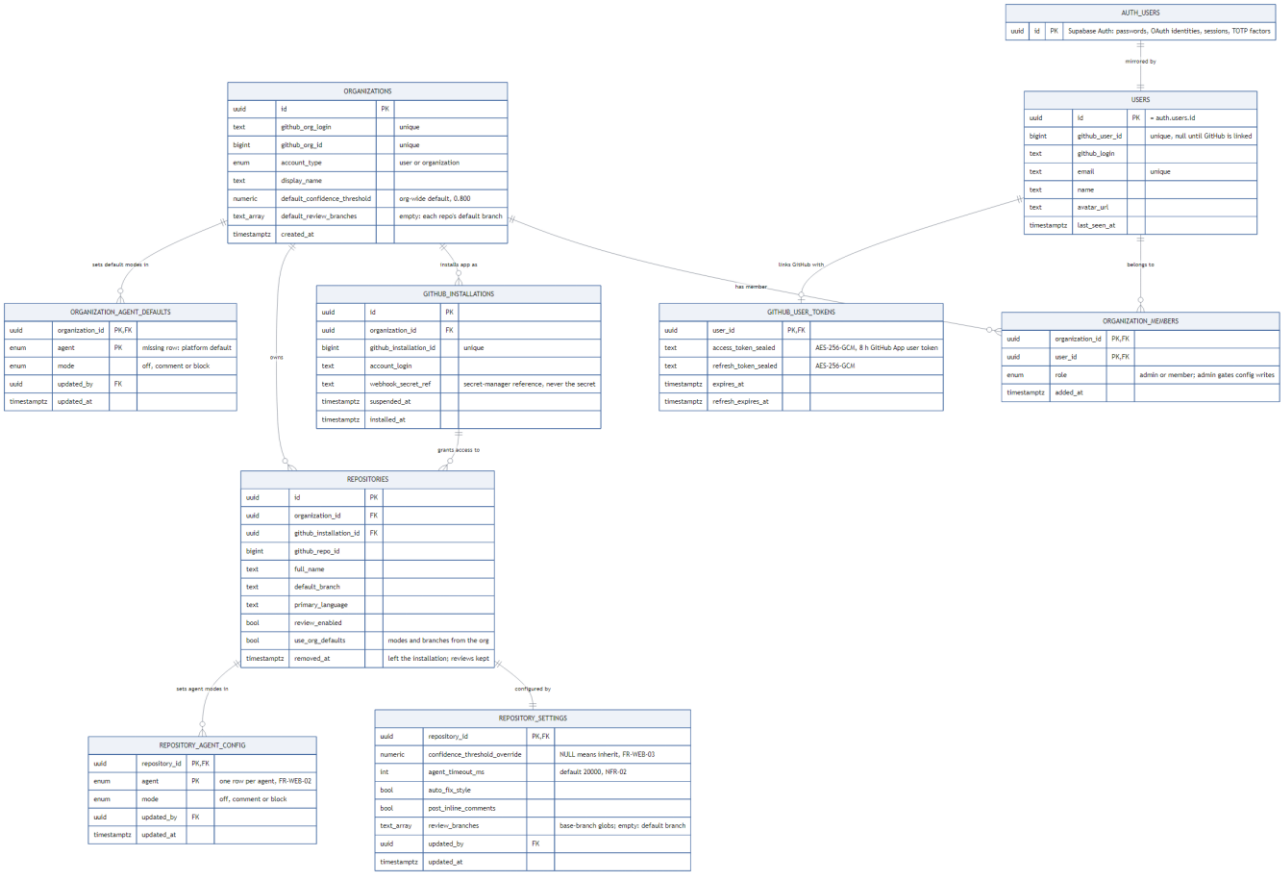


Figure 3.1 Identity, installations and per-repository configuration

3.3 Review pipeline

- reviews one review job, with trigger, commit SHAs, resolved confidence threshold, status and GitHub check-run linkage. A unique index on (repository, pr_number, head_sha) stops a redelivered webhook starting a duplicate job.
- agent_runs one row per agent invoked, written even on timeout, so per-agent latency, LLM provider, model and failure reason (including quota deferral) are always recorded.
- findings a single issue anchored to a file and line range, with severity, confidence and severity_rank. dedupe_key = hash(file_path, line range, rule/issue type) collapses the same issue reported by two agents.
- suggestions a proposed fix (deterministic or AI-suggested). An AI fix cannot be marked accepted without a resolving user.
- finding_embeddings / finding_similarities the pgvector embedding of each finding and the similarity scores retrieved for review context.

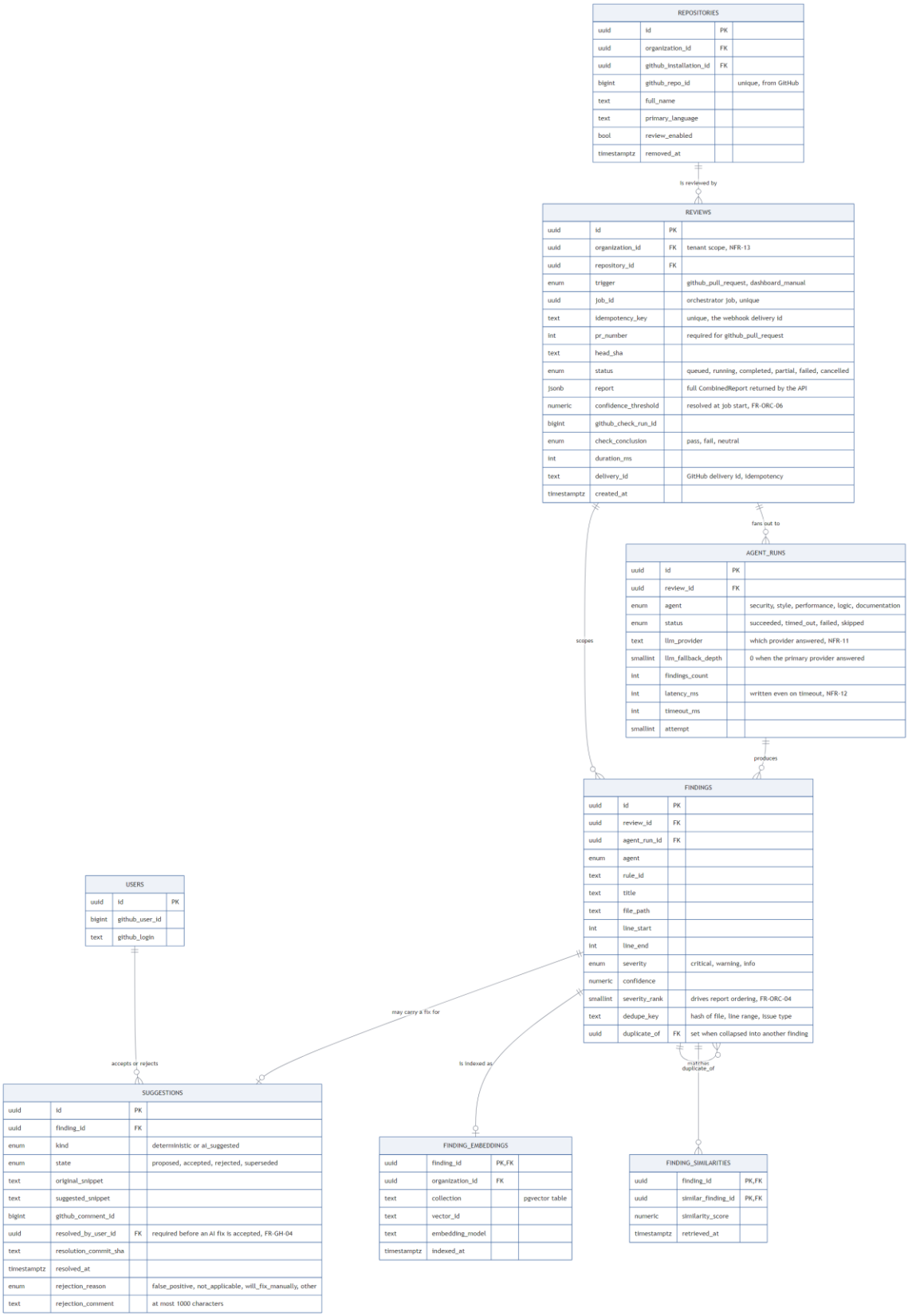


Figure 3.2 Reviews, agent runs, findings, suggestions and vector bookkeeping

4. Class, State and Sequence Diagrams

4.1 Orchestrator

The orchestrator is a LangGraph StateGraph. A ReviewState flows through fan-out, aggregate, context and threshold nodes and is persisted through a single repository. A rejected or timed-out agent yields status timed_out rather than failing the graph.

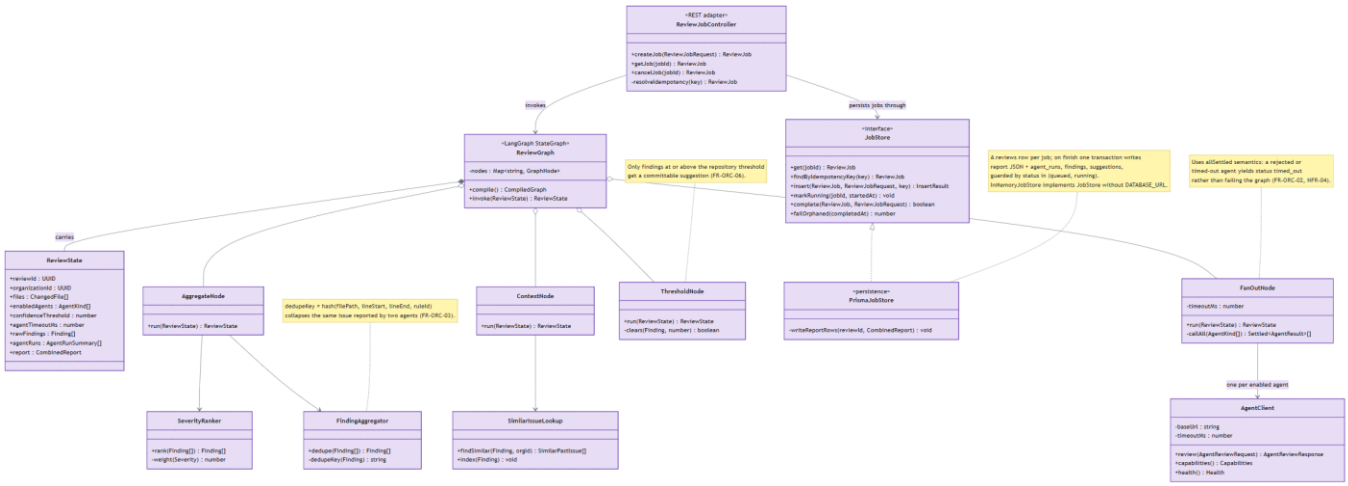


Figure 4.1 Orchestrator class diagram

4.2 Agent services

All five services implement one ReviewAgent contract. BaseAgentService is abstract; each concrete agent supplies its own analyze(). The Style Agent is the only agent with no LlmClient its fixes are rule-based and run inside a network-disabled Docker sandbox.

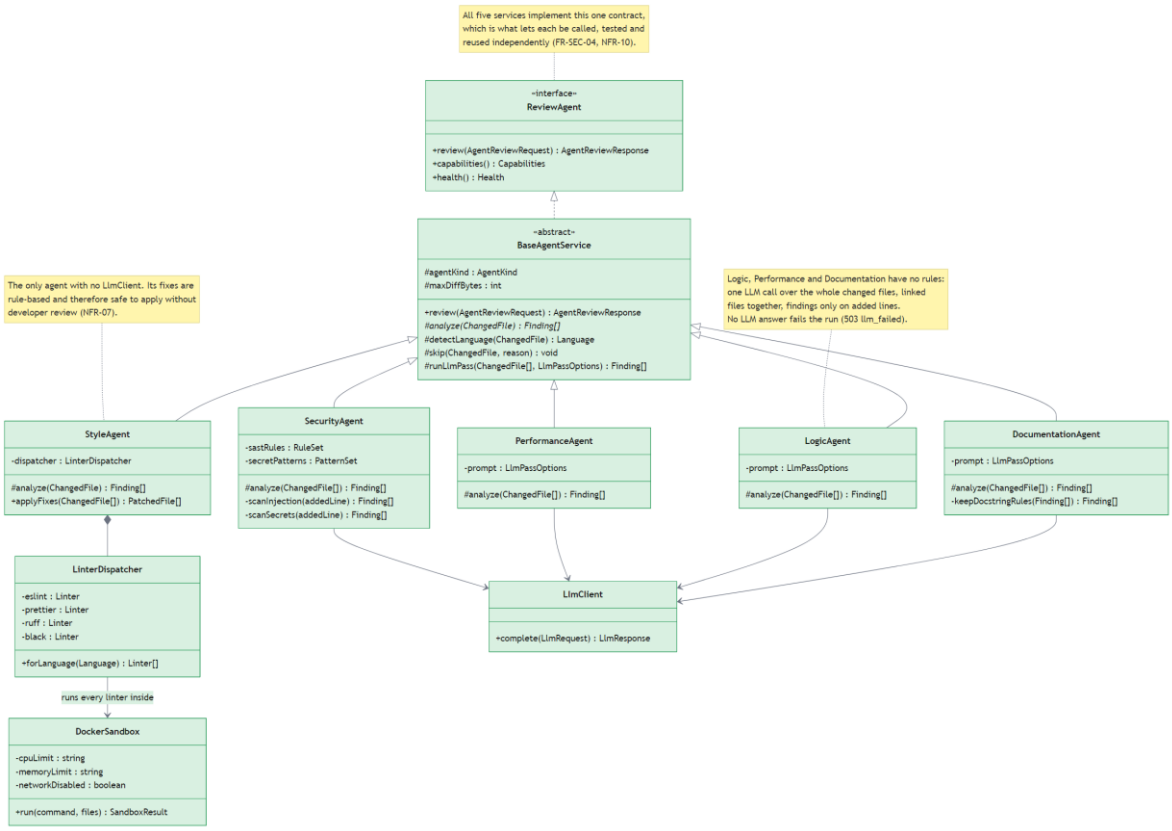


Figure 4.2 Agent service class diagram

4.3 LLM client, quota budget and large diffs

Groq is the primary LLM provider and Gemini the secondary. Groq runs openai/gpt-oss-120b and Gemini runs Gemini 3.5 Flash Lite; gemini-embedding-2 produces the 768-dimension vectors stored in pgvector, because Groq offers no embedding model. A batch goes to Groq when Groq's quota can take it now, otherwise to Gemini, and a Groq call that fails (rate limit, server error or timeout) is retried once on Gemini, recorded as fallback depth 1. Development and production use separate API keys, so local testing never consumes the production quota.

Production must never receive a rate-limit error, so the orchestrator reserves quota before sending each batch instead of retrying afterwards. One QuotaBudget for Gemini and one per Groq account (Groq limits apply per account, so each LLM agent may use its own account, named in GROQ_ACCOUNT_<AGENT>) track requests and tokens over a sliding one-minute window and a rolling 24-hour window, and allow only 80% of each key's free-tier limits: Groq 30 RPM, 8K TPM, 1K RPD and 200K tokens per day; Gemini 15 RPM, 250K TPM and 500 RPD. Groq's 8K tokens per minute caps a Groq call at about 3.4K tokens, so small reviews stay on Groq and larger batches go to Gemini. A batch that fits neither provider waits in the queue; if it still cannot fit before its deadline, the LLM agent is marked skipped with llm_quota_exhausted and the review completes as partial. Reservations are settled with the token counts the answering provider reports.

Agents read each changed file whole, with the lines the pull request added marked, and report findings only on added lines. The gateway drops binary, generated and vendored files and attaches the full text of each remaining file up to 36 KB. For each LLM agent the orchestrator groups files that import one another into one LLM call, so a change is read together with the changed code it uses; every call also carries the rest of the pull request's diffs as context (up to 4K tokens). A file over 12K estimated tokens is sent as its diff only, a group over 40K tokens sends its largest files as diffs until it fits, and files beyond 80K tokens per agent are over_budget. Files are never split. A typical review costs one call per LLM agent. The combined report lists every skipped file with its reason and a coverage summary, so a partial review states its gap rather than hiding it.

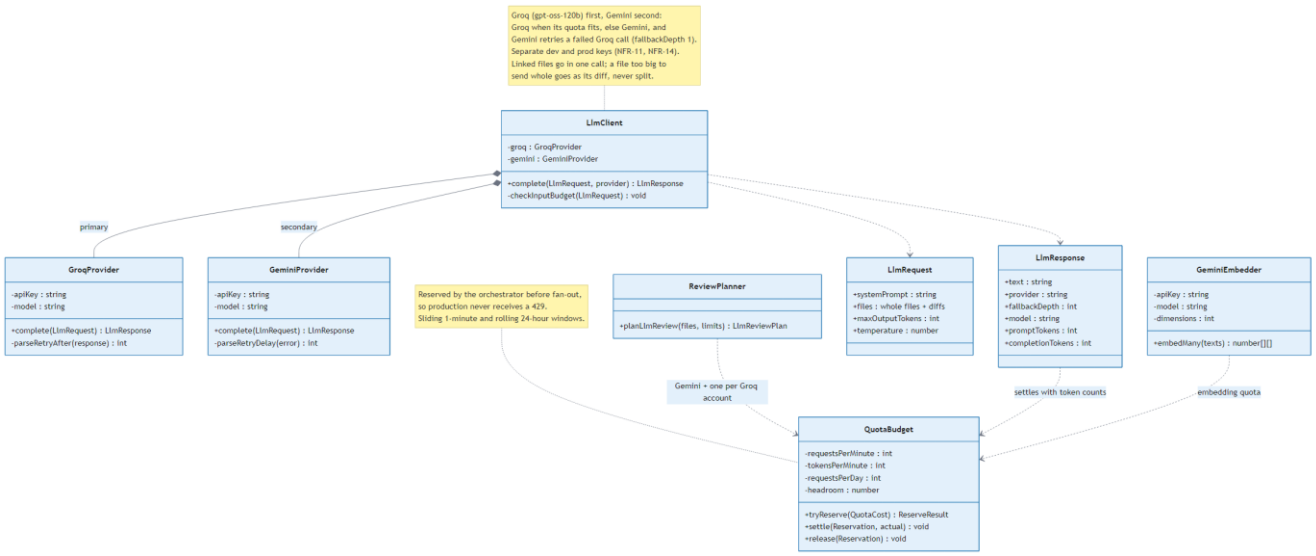


Figure 4.3 LLM client class diagram

4.4 Review job lifecycle

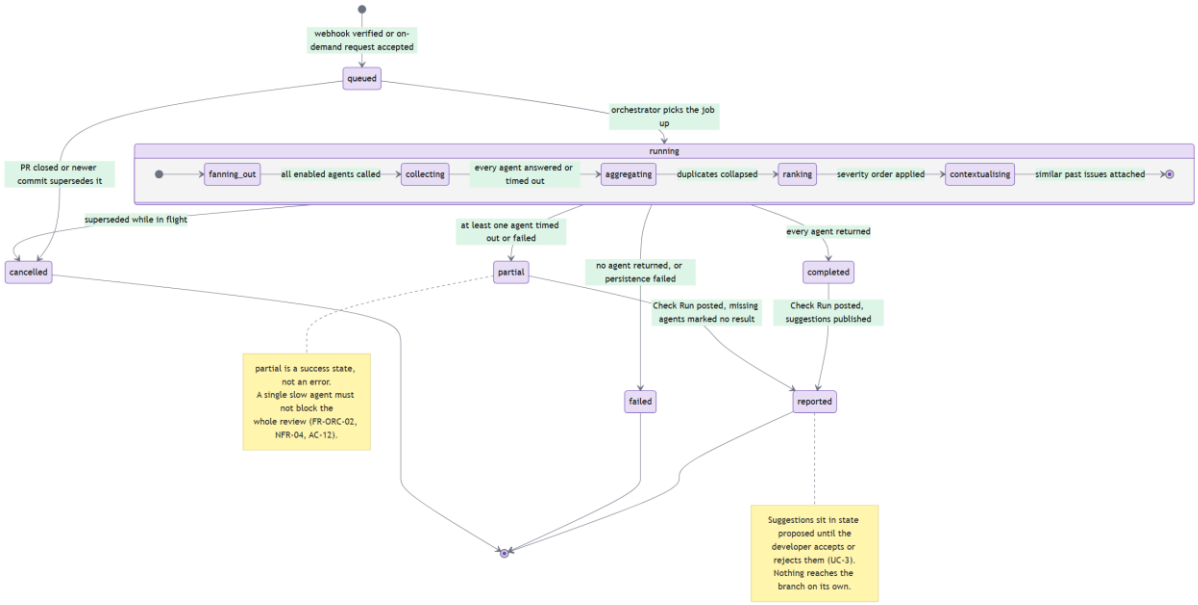


Figure 4.4 Review state machine. partial is a success state: a single slow agent must not block the whole review.

4.5 Sequence diagrams

UC-1 Automatic pull request review. A verified webhook triggers an asynchronous fan-out; a Check Run is created within five seconds; the timed-out agent is marked "no result" and the review still completes as partial.

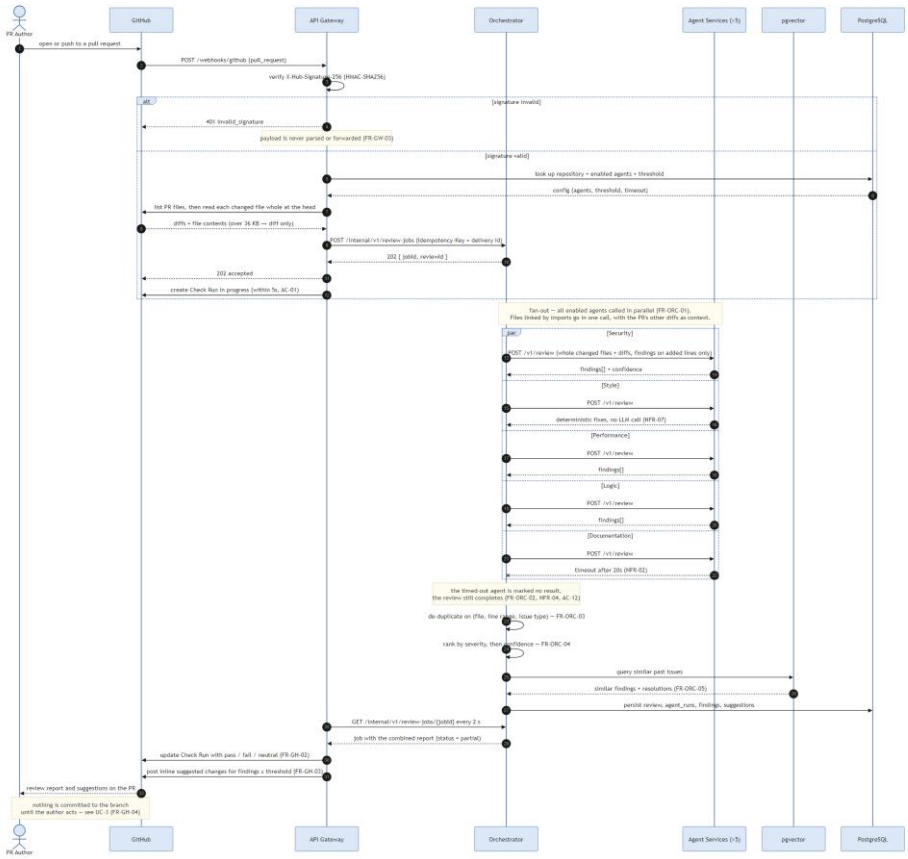


Figure 4.5 UC-1 automatic pull request review

UC-2 On-demand review from the dashboard. A member clicks Re-run review on an open pull request; the gateway reviews its head commit through the same path as a webhook (trigger dashboard_manual), and the review page refreshes until the report is complete.

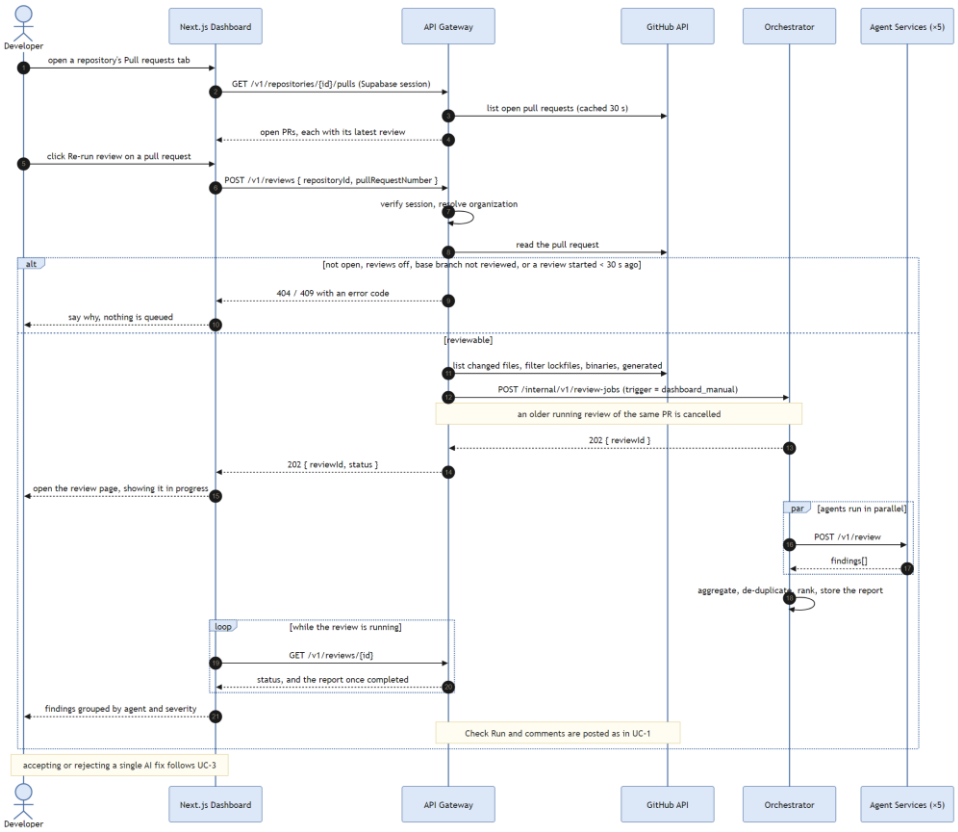


Figure 4.6 UC-2 on-demand review from the dashboard

UC-3 Commit or dismiss a suggested fix from the dashboard. Commit fix needs a recent authenticator code (TOTP step-up) and writes the fix to the pull request branch only if the branch head and the reviewed lines are unchanged and the branch is not a fork; Dismiss records a reason. Committing is the only path to a branch commit; a fix already decided or superseded by a newer push returns 409.

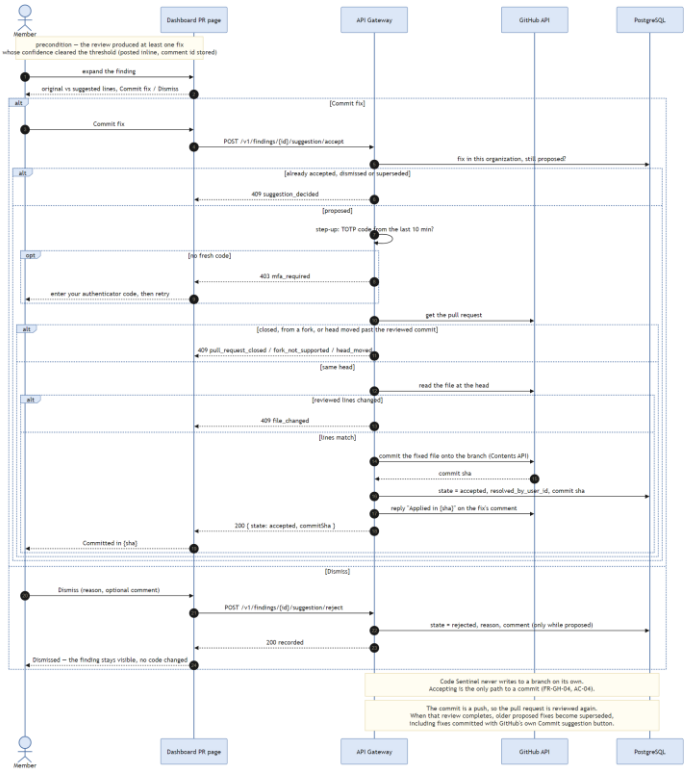


Figure 4.7 UC-3 commit or dismiss a suggested fix from the dashboard

UC-4 Configure repository and agent settings. Admin-only; disabling every agent is rejected; the change applies from the next review job, not retro-actively.

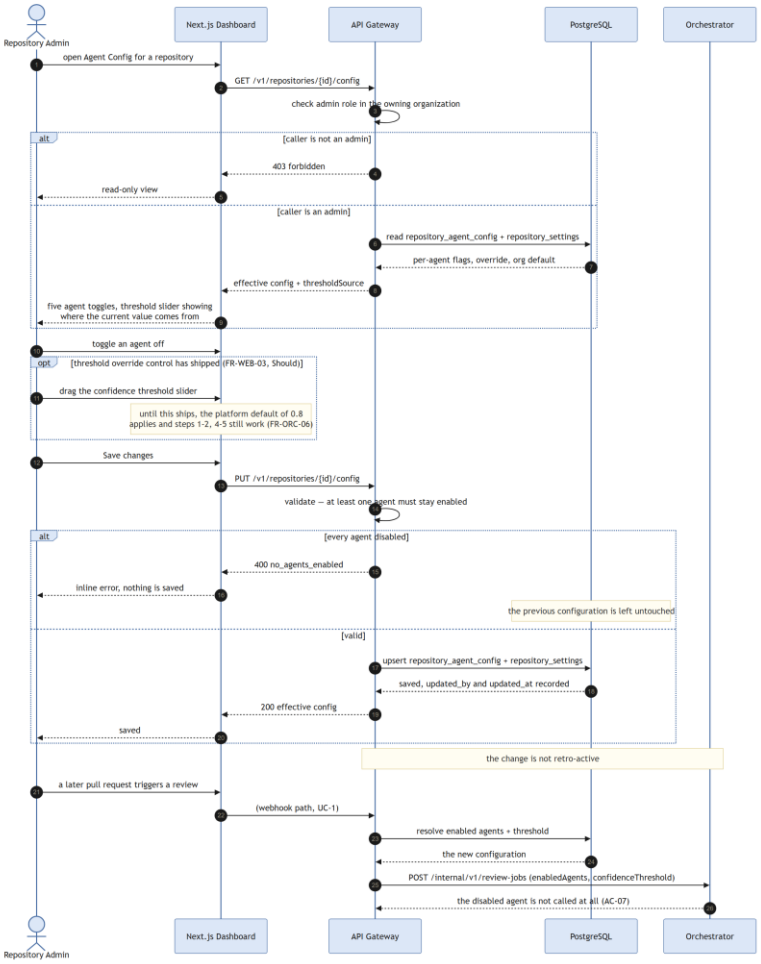


Figure 4.8 UC-4 configure repository and agent settings

UC-5 Browse review history. Every query is scoped to the caller’s organization; no matching reviews produces an empty state rather than an empty table.

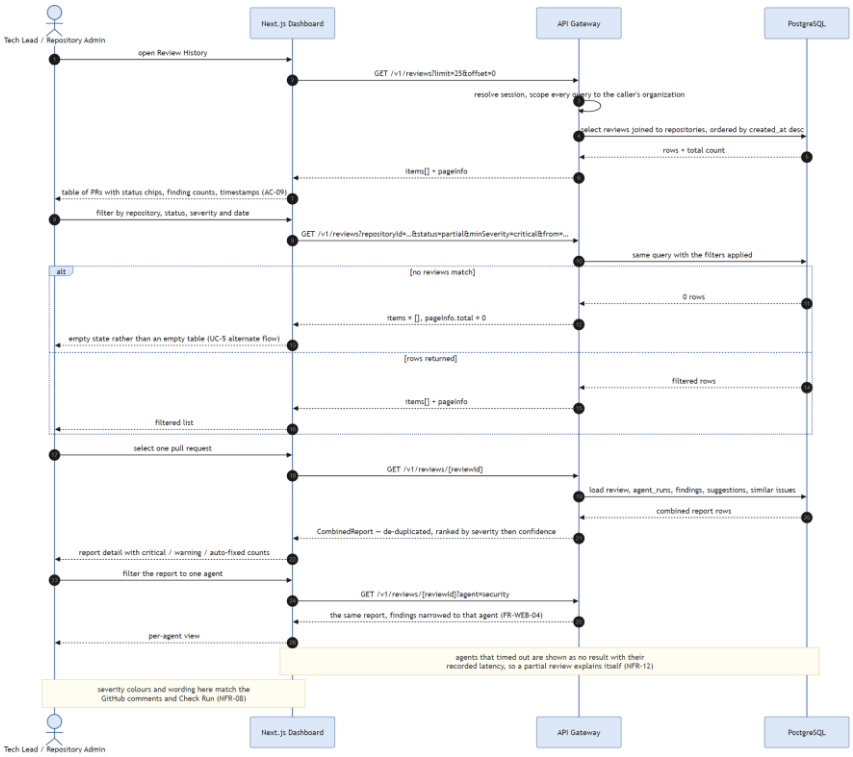


Figure 4.9 UC-5 browse review history

5. REST API Contracts (OpenAPI / Swagger)

Three OpenAPI 3.1 documents describe the system, plus a shared schema file (common.yaml) so that a Finding produced by an agent, carried by the orchestrator and rendered by a client is described exactly once.

- gateway.yaml the single public entry point for both clients, the GitHub App and the dashboard.
- orchestrator.yaml internal contract between the gateway and the LangGraph orchestrator; not exposed to the internet.
- agent.yaml one contract implemented five times by the agent services.
- common.yaml shared schemas: Finding, Suggestion, CombinedReport, AgentRunSummary, ChangedFile, SkippedFile, ReviewCoverage, Severity, Confidence, and the standard error responses.

5.1 API Gateway (gateway.yaml)

Base URL https://api.codesentinel.dev. Auth: GitHub webhooks by HMAC-SHA256 signature; dashboard by a Supabase Auth access token (bearer, verified against Supabase's signing keys), with an authenticator-code step-up on configuration changes and on committing fixes. Every non-webhook read is organization-scoped.

Method & path	Purpose	Use case
POST /webhooks/github	Receive a GitHub webhook event (signature verified before the body is read)	UC-1
POST /v1/reviews	Re-run the review of an open pull request	UC-2
GET /v1/reviews	List review history, filterable by repo, status, severity, trigger, date	UC-5
GET /v1/reviews/{reviewId}	Get the combined, de-duplicated, ranked report	UC-2, UC-5
GET /v1/reviews/{reviewId}/events	Stream review progress (SSE)	UC-2
POST /v1/findings/{findingId}/suggestion/accept	Commit a suggested fix to the PR branch (TOTP step-up)	UC-3
POST /v1/findings/{findingId}/suggestion/reject	Dismiss a suggested fix, with a reason and optional comment	UC-3
GET /v1/repositories	List repositories CodeSentinel is installed on	UC-4
GET /v1/repositories/{repositoryId}/config	Read effective agent configuration and where each value came from	UC-4
PUT /v1/repositories/{repositoryId}/config	Update agent configuration (admin only; at least one agent must stay enabled)	UC-4
GET /v1/me	Identify the caller and list organization memberships	Account
GET /healthz	Liveness and readiness probe	Service

5.2 Orchestrator (orchestrator.yaml)

Base URL http://orchestrator.internal:8080. Auth: short-lived service bearer token. A review job is accepted immediately and runs asynchronously because a full fan-out can take up to a minute.

Method & path	Purpose
POST /internal/v1/review-jobs	Start a review and fan out to the enabled agents; Idempotency-Key (GitHub delivery id) makes a redelivery harmless
GET /internal/v1/review-jobs/{jobId}	Poll a job and read its combined report; agentRuns shows which agents have answered
POST /internal/v1/review-jobs/{jobId}/cancel	Cancel a running job when the PR is closed or superseded by a newer commit
GET /internal/v1/agents/health	Report the last known health of each agent service
GET /healthz	Liveness and readiness probe

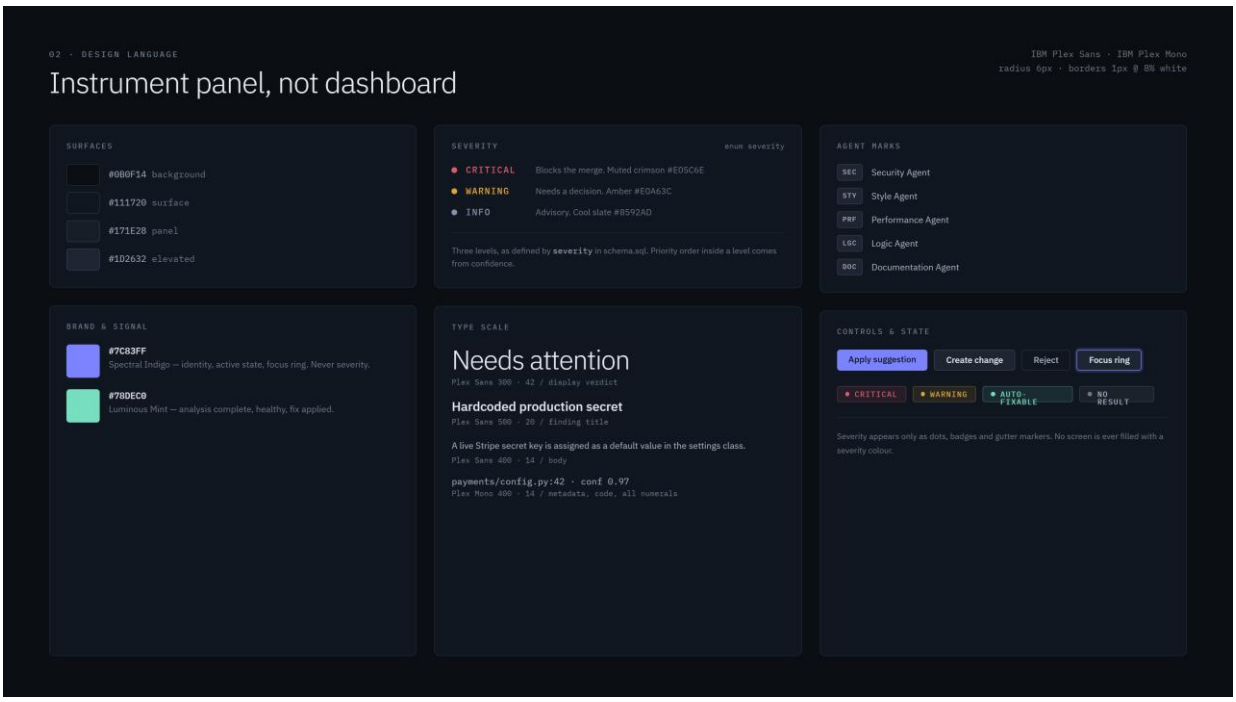
5.3 Agent service (agent.yaml)

One contract, implemented five times (ports 8081–8085). Auth: short-lived service bearer token. An agent receives the changed files (whole when available) with their diffs and the rest of the pull request as context, returns findings on added lines, and is otherwise stateless; it never calls another agent and never writes to PostgreSQL.

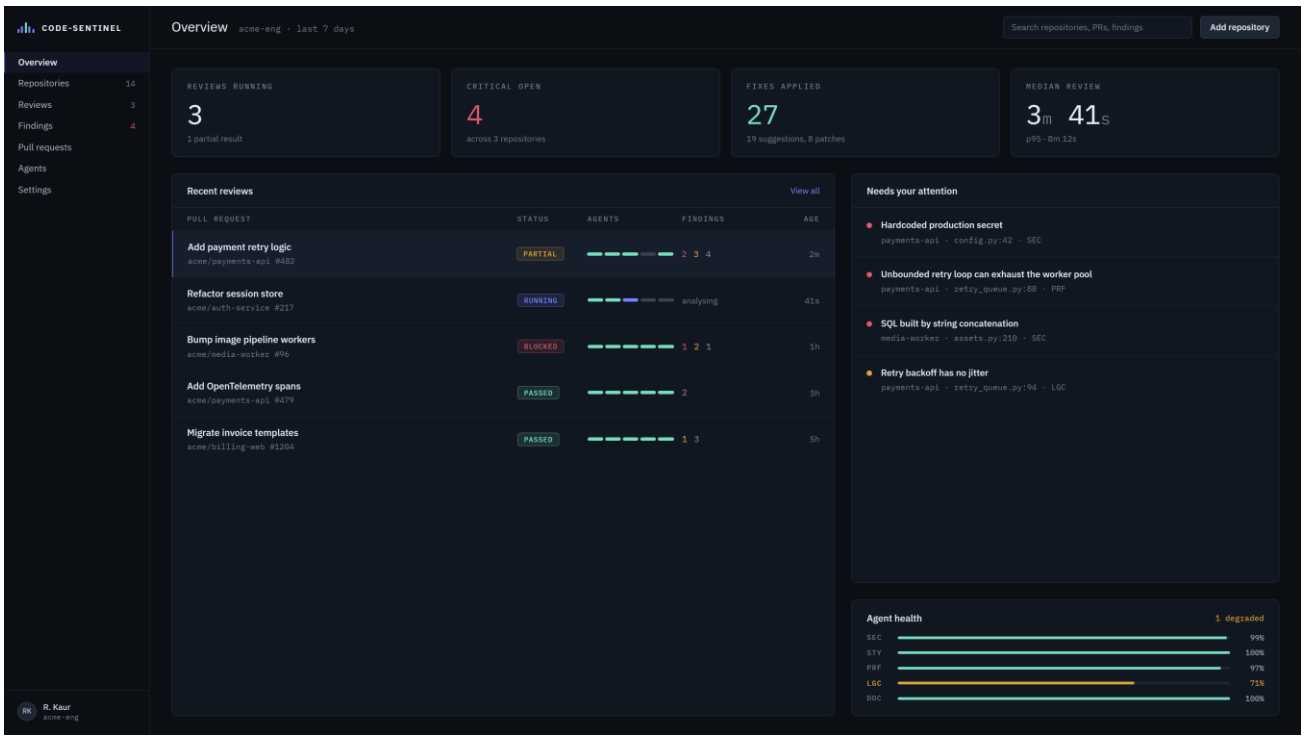
Method & path	Purpose
POST /v1/review	Analyze changed files and return findings with confidence; an empty findings array is a valid, successful result; files over the agent's size limit are returned as skipped, never split
GET /v1/capabilities	Describe the languages, rules and size limits (maxFileTokens) this agent supports; lets the orchestrator skip unsupported languages
GET /healthz	Liveness and readiness probe

6. Wireframes / UI-UX Mockups

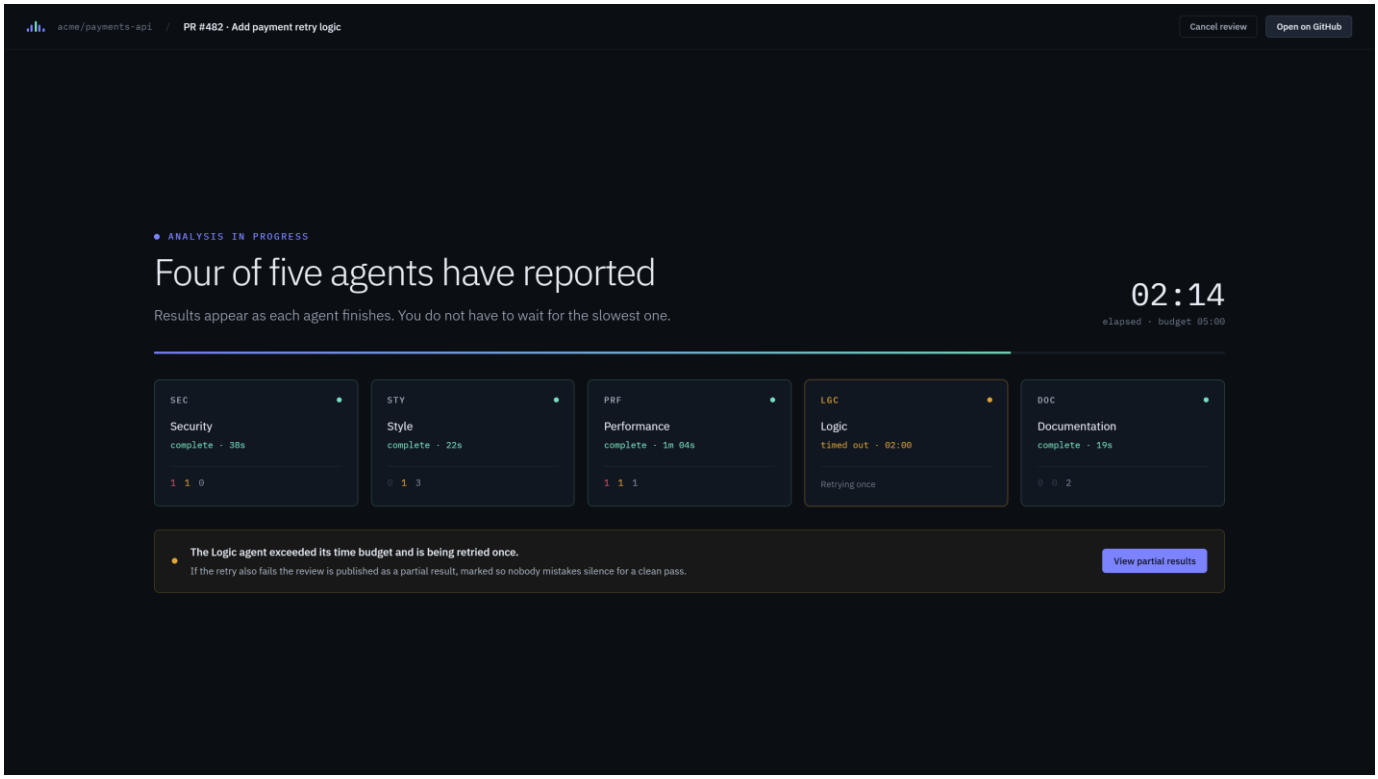
14 frames on a 1920×1080 design canvas (exported at 2×). Every screen shares one scenario: pull request #482 "Add payment retry logic" on acme/payments-api. The run is partial the Logic agent exceeded its 90-second budget, the other four completed, and the report ships with the gap stated rather than hidden. Severity scale: critical / warning / info; ordering within a severity level comes from the agent’s confidence score.



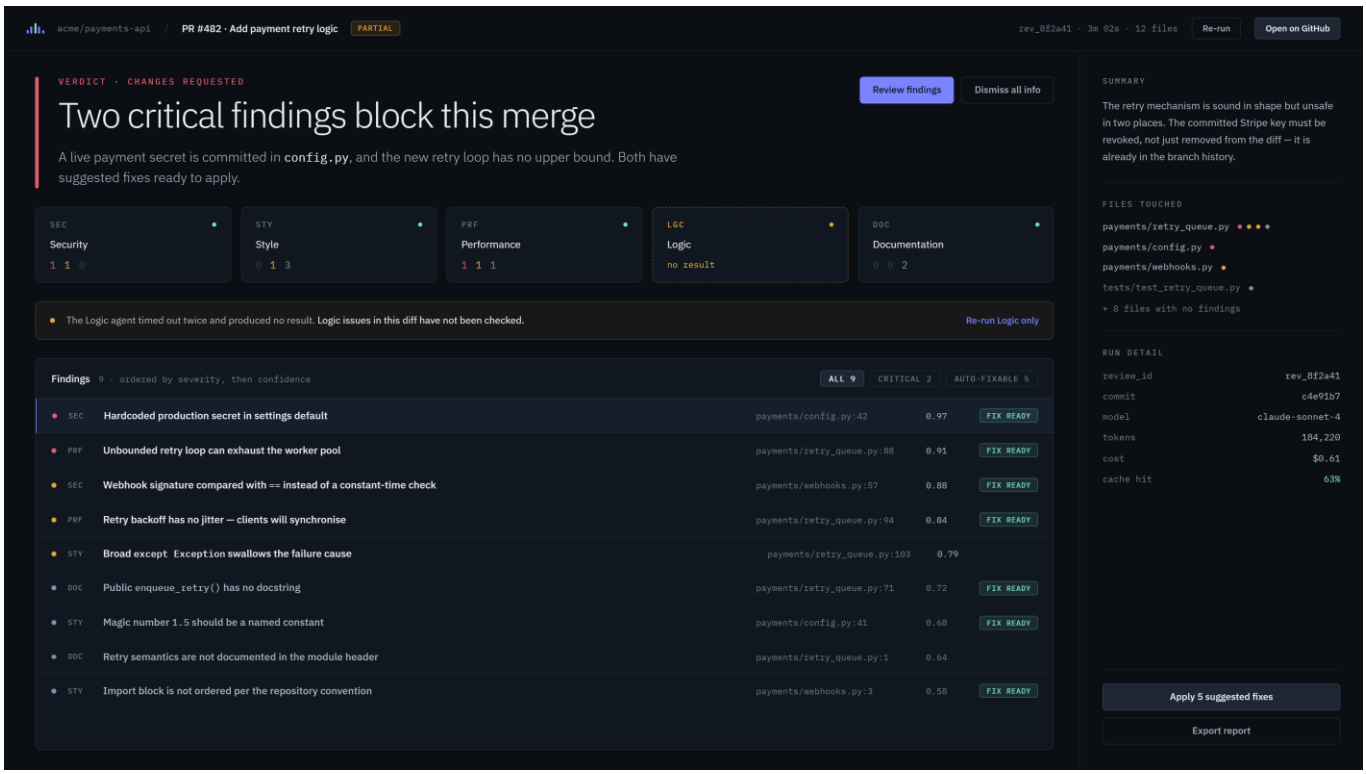
Frame 02 Design language: surfaces, brand, severity colours, type scale, agent marks, controls.



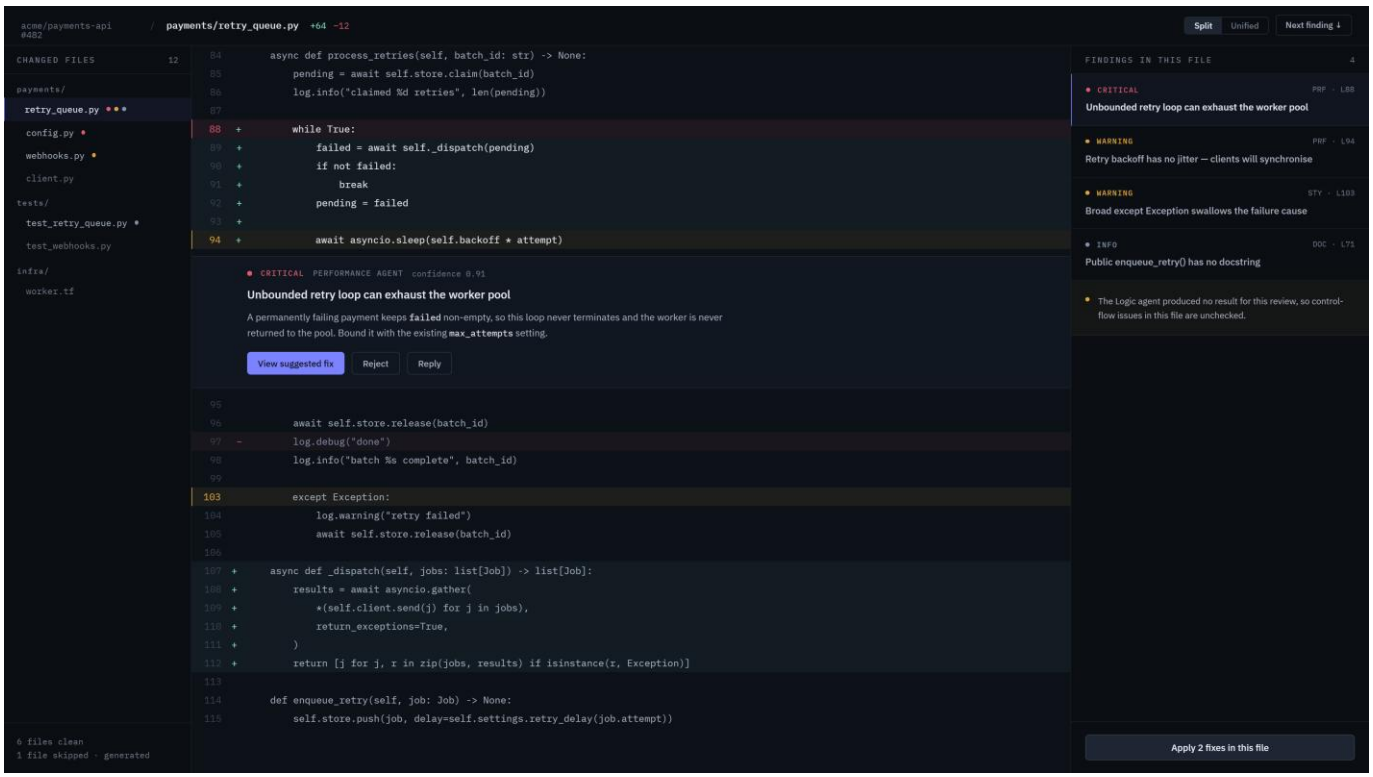
Frame 03 Dashboard: cross-repo view of what needs the user across 14 repositories, ordered by blocked merges.



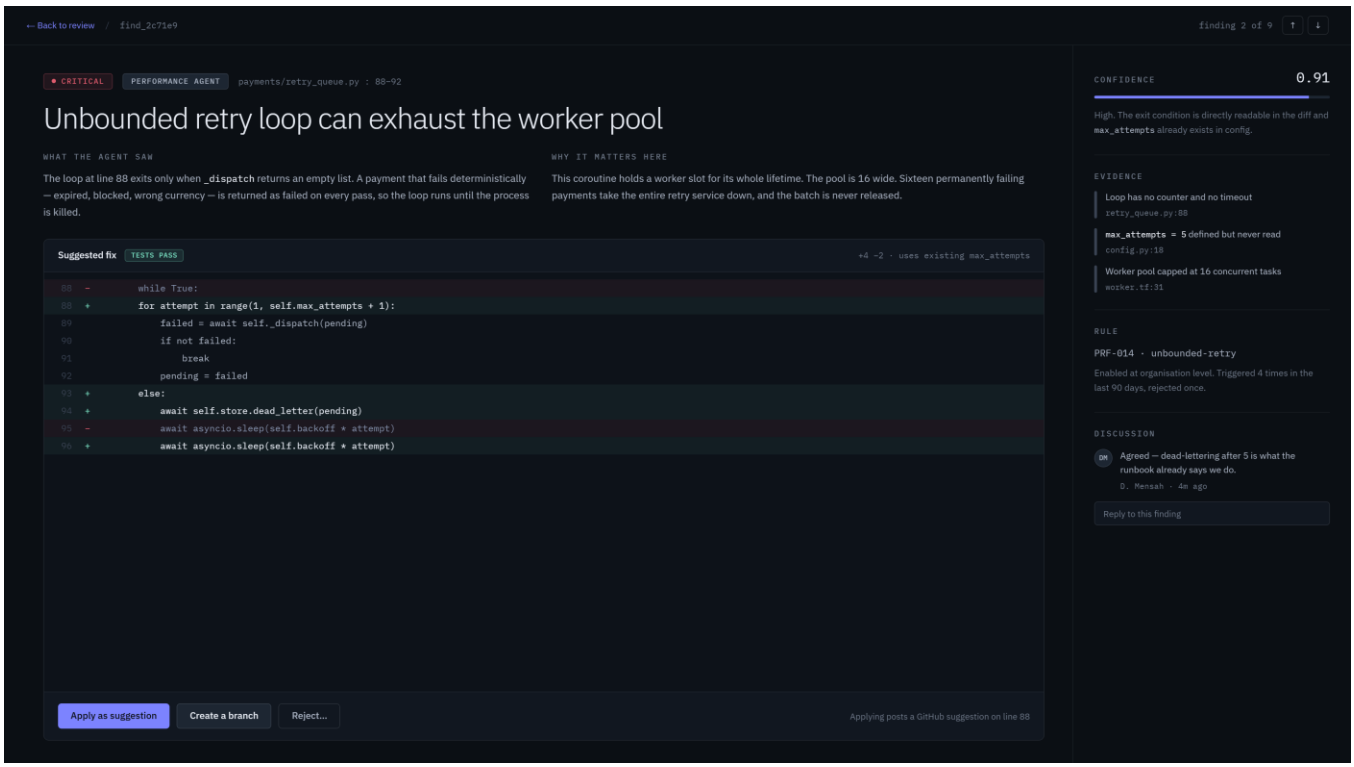
Frame 04 Review in progress: four agents returned, Logic is over budget, findings are already readable.



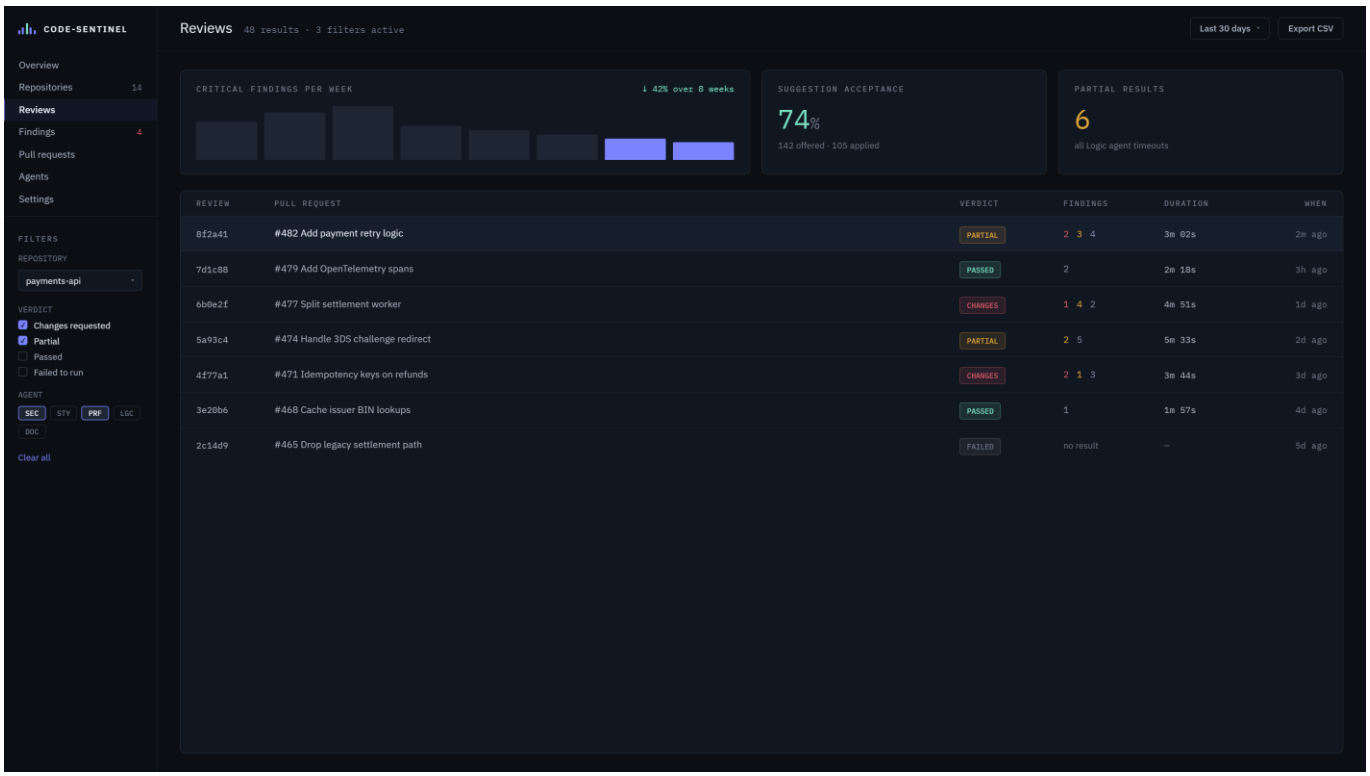
Frame 05 Review overview: the verdict, the five-agent matrix, and the priority queue. Answers "can I merge this".



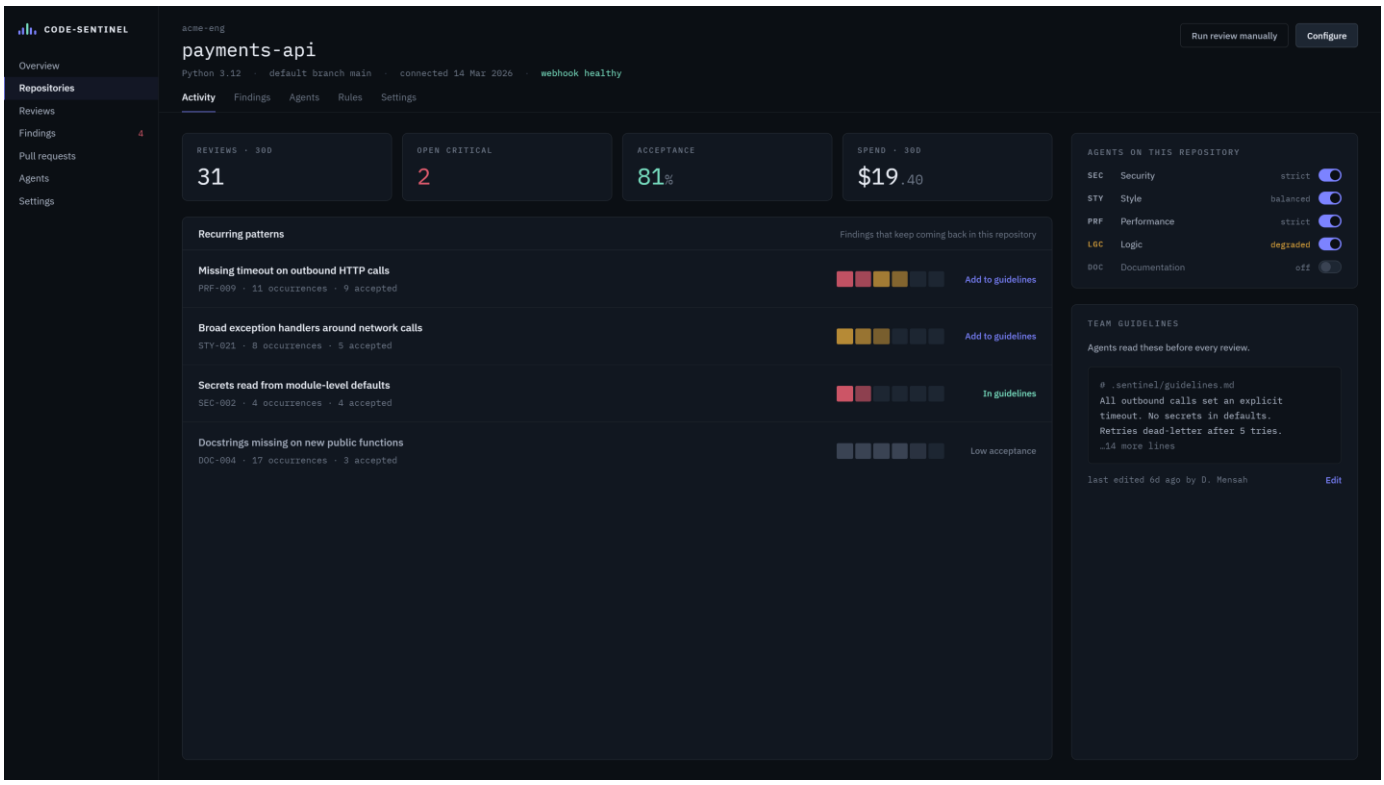
Frame 06 Three-panel code review: files / diff / findings, with findings anchored to the line that produced them.



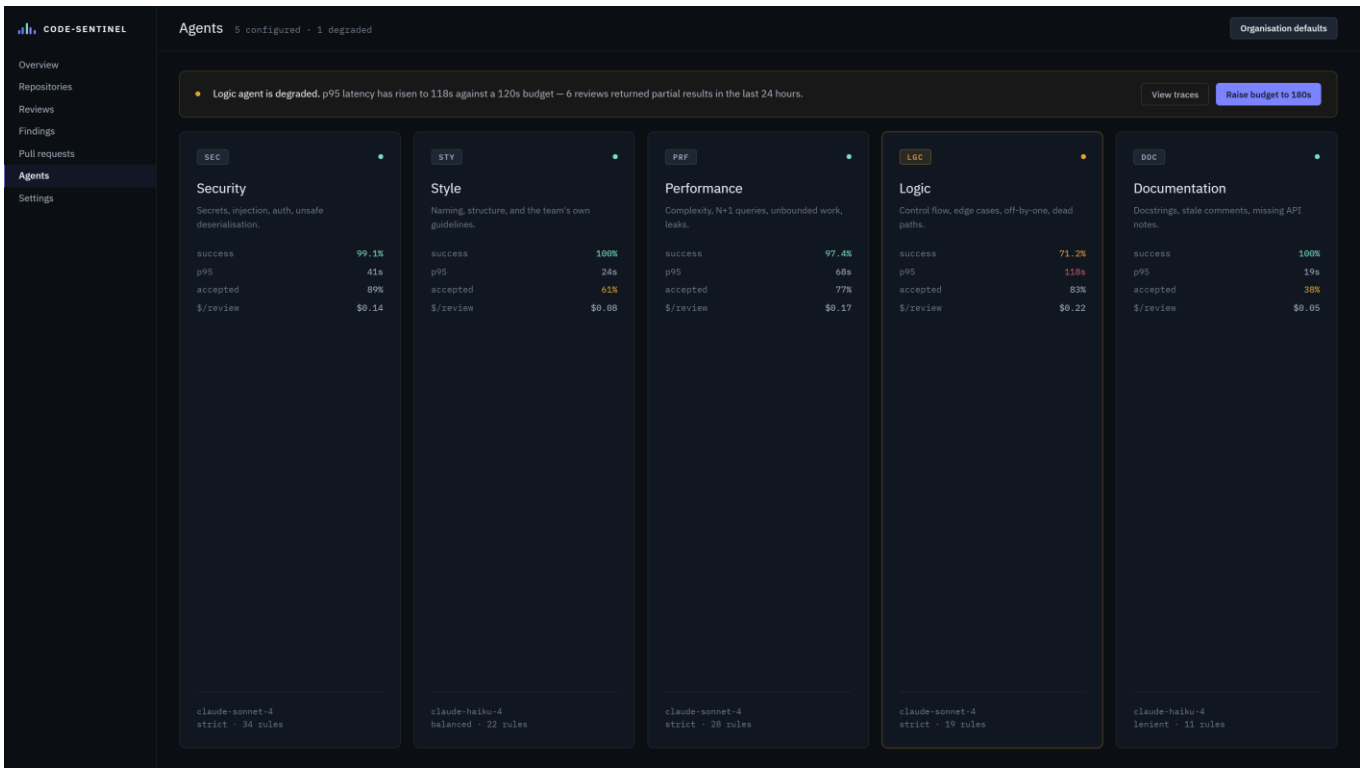
Frame 07 Finding detail & fix: evidence, reasoning, proposed patch, confidence, rule provenance. Apply, branch, or reject with a reason.



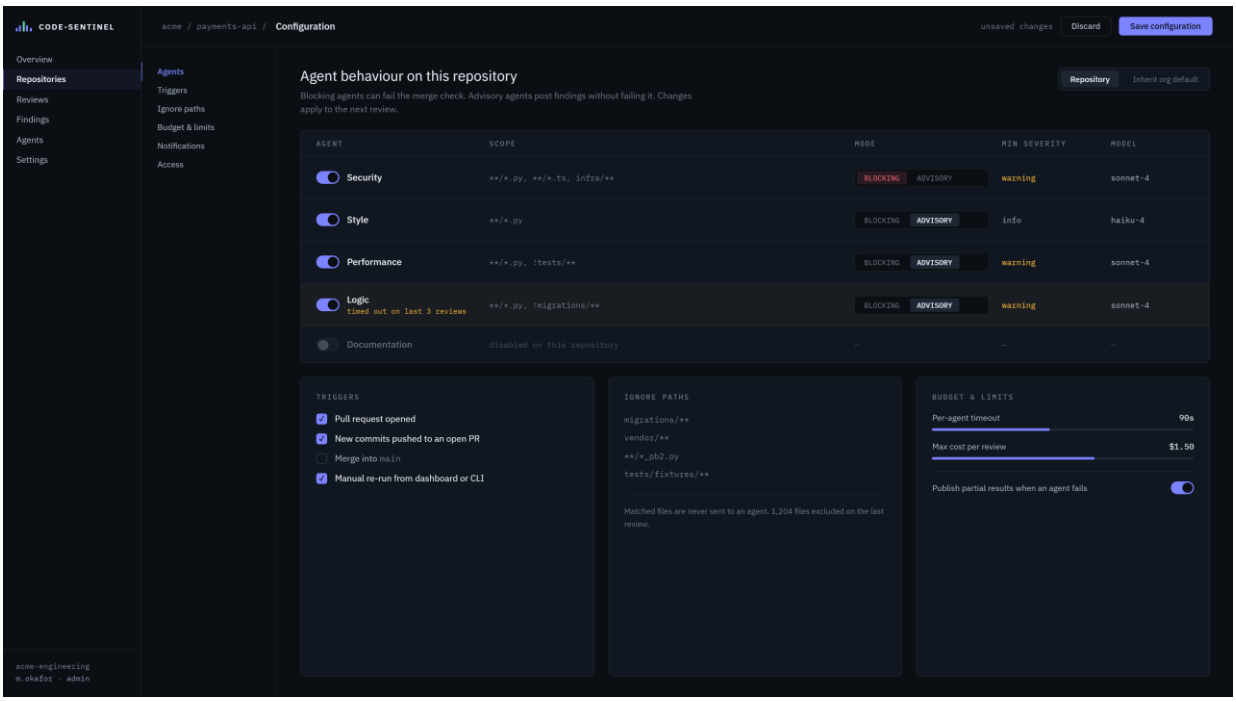
Frame 08 Review history: filterable record of past reviews. Answers "is the codebase getting better".



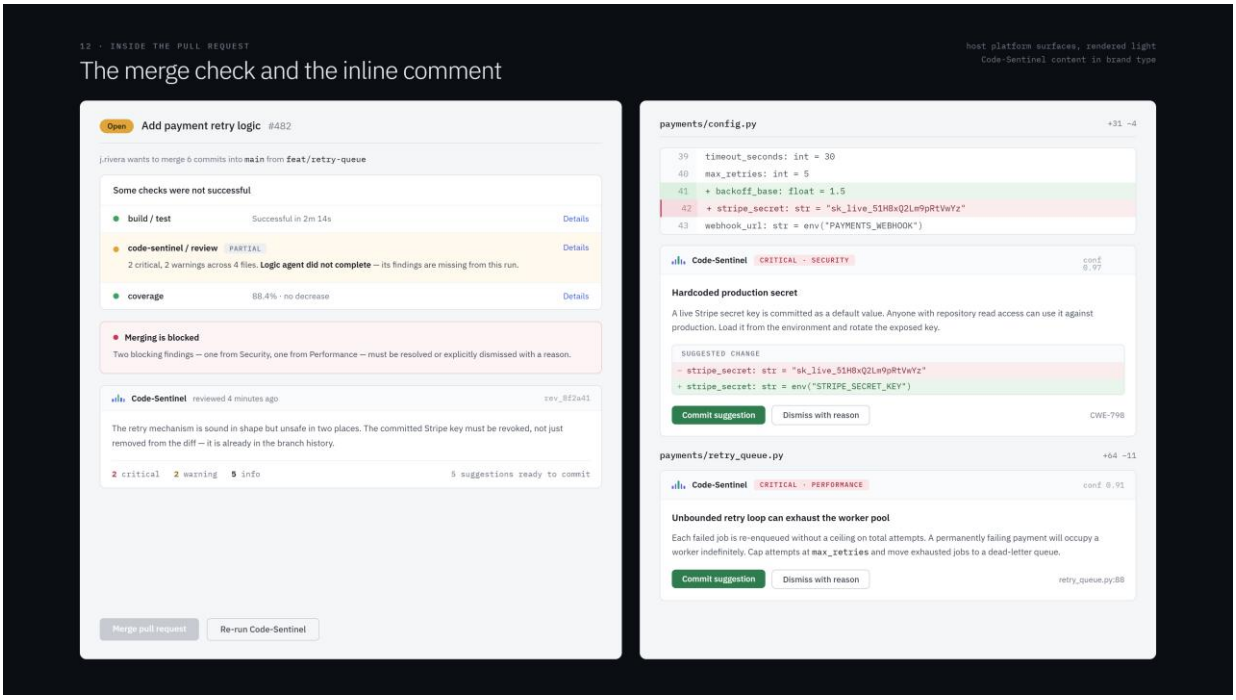
Frame 09 Repository detail: active agents, recurring problems, and where the repository's rules came from.



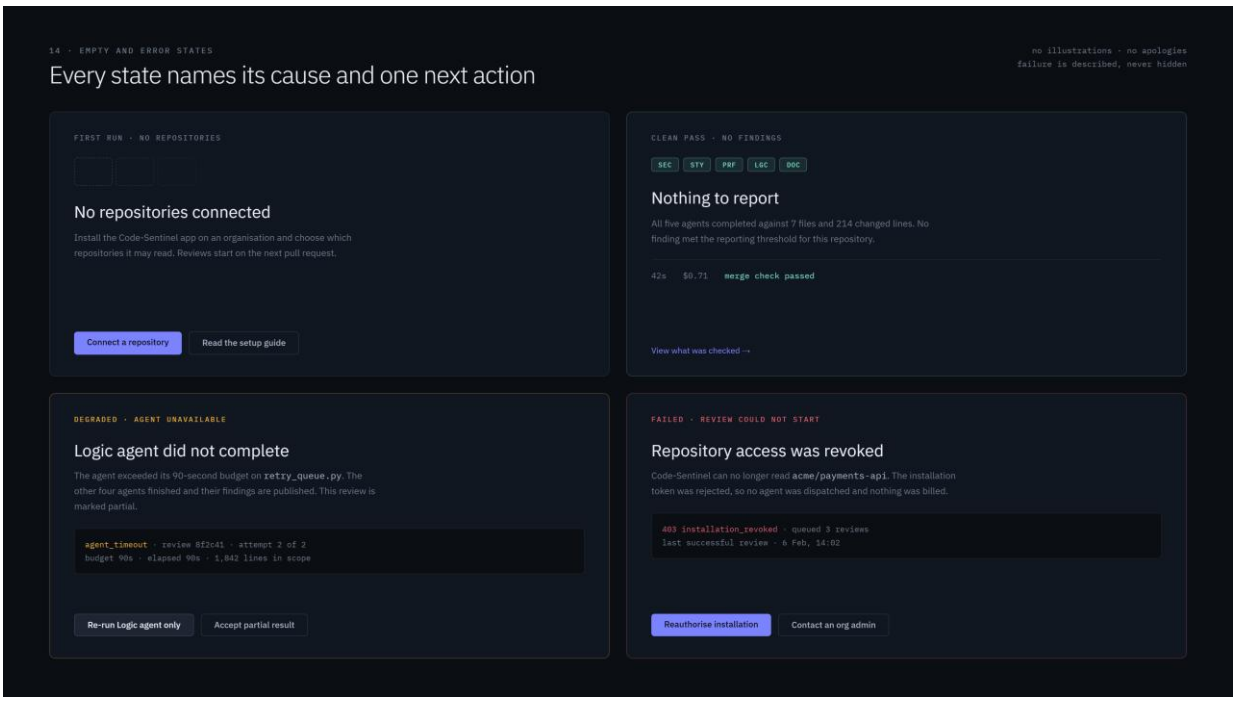
Frame 10 Agents as operational objects: success rate, p95 latency, acceptance rate, cost per review.



Frame 11 Repository & agent configuration: per-agent mode, scope, thresholds, triggers, ignore paths, budgets.



Frame 12 Pull request check: the merge check and two inline comments, where most developers meet the product.



Frame 14 Empty & error states: first run, clean pass, degraded run, revoked access. Each names its cause and offers one next action.